



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

Институт информационных технологий (ИТ)

Кафедра математического обеспечения и стандартизации информационных технологий

ОТЧЁТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5

по дисциплине «Структуры и алгоритмы обработки данных»

Выполнил студент группы **ИКБО-50-25**

Багинян А.В.

Принял *преподаватель*

Рысин М.Л.

Москва 2026

ПРАКТИЧЕСКАЯ РАБОТА №5

«Работа с данными из файла»

Часть 5.1. Алгоритмы поиска в таблице при работе с данными из файла

Вариант 4

Цель: получить практический опыт по применению алгоритмов поиска в таблицах данных.

Вариант 4. Алгоритм поиска — поиск Фибоначчи. Структура записи — владелец автомобиля: номер машины, марка, сведения о владельце.

1. Задание 1

1.1. Постановка задачи

Создать двоичный файл из записей «Владелец автомобиля». Поле ключа (номер машины) заполнить датчиком случайных чисел, ключи уникальны. Файл создаётся через промежуточный текстовый файл.

1.2. Описание подхода к решению

Структура записи:

```
1 struct Car {  
2     char number[9]; // госномер, например "A123BC77"  
3     char brand[16]; // марка  
4     char owner[48]; // сведения о владельце (ФИО)  
5 };
```

Размер записи: $9 + 16 + 48 = 73$ байта (все поля типа `char`, выравнивание не добавляет байтов).

Прямой доступ: все записи имеют одинаковый размер, поэтому запись с номером i начинается со смещения $i \cdot \text{sizeof}(\text{Car})$ байт. Указатель чтения устанавливается методом `seekg(offset)`, затем запись считывается целиком методом `read((char*)&car, sizeof(Car))`.

Алгоритмы, реализованные в форме функций:

- генерация текстового файла со случайными уникальными ключами;
- преобразование текстового файла в двоичный;
- определение числа записей в файле;
- чтение записи по номеру (прямой доступ);
- вывод записи;
- линейный поиск записи в файле (задание 2);
- построение таблицы «ключ — смещение», поиск Фибоначчи в таблице, чтение записи по смещению (задание 3).

Прототипы функций (car.h):

```
1  #pragma once
2
3  #include <fstream>
4  #include <string>
5  #include <vector>
6
7  // Запись файла: владелец автомобиля. Ключ - номер машины.
8  struct Car {
9      char number[9]; // госномер, например "A123BC77"
10     char brand[16]; // марка
11     char owner[48]; // сведения о владельце (ФИО)
12 };
13
14 // Элемент таблицы: ключ и смещение записи в файле
15 struct IndexEntry {
16     char number[9];
17     long offset;
18 };
19
20 // Задание 1
21 void generateTextFile(const std::string& path, int n, unsigned seed);
22 void textToBinary(const std::string& txtPath, const std::string&
binPath);
23 long recordCount(std::ifstream& file);
24 Car readRecord(std::ifstream& file, long index);
25 void printCar(const Car& car);
26
27 // Задание 2
28 long linearSearch(std::ifstream& file, const char* key, Car& result);
29
30 // Задание 3
31 std::vector<IndexEntry> buildTable(std::ifstream& file);
32 long fibonacciSearch(const std::vector<IndexEntry>& table, const char*
key);
33 Car readByOffset(std::ifstream& file, long offset);
34
```

1.3. Код программы

```
1  // Предусловие: n > 0, файл path доступен для записи.
2  // Постусловие: в path записано n строк "номер;марка;владелец" с
уникальными
3  // случайными номерами.
4  void generateTextFile(const std::string& path, int n, unsigned seed) {
5      std::mt19937 rng(seed);
6      auto rnd = [&](int m) { return (int)(rng() % m); };
7  }
```

```

7     std::set<std::string> used;
8     std::ofstream out(path);
9     while ((int)used.size() < n) {
10         char number[9];
11         snprintf(number, sizeof number, "%c%03d%c%c%02d",
LETTERS[rnd(12)], rnd(1000),
12             LETTERS[rnd(12)], LETTERS[rnd(12)], 1 + rnd(99));
13         if (!used.insert(number).second) continue; // ключ уже есть
14         out << number << ';' << BRANDS[rnd(10)] << ';'
15             << SURNAMES[rnd(10)] << ' ' << INITIALS[rnd(6)] << '\n';
16     }
17 }

```

```

1 // Предусловие: txtPath - файл, созданный generateTextFile.
2 // Постусловие: binPath содержит те же записи в виде структур Car.
3 void textToBinary(const std::string& txtPath, const std::string&
binPath) {
4     std::ifstream in(txtPath);
5     std::ofstream out(binPath, std::ios::binary);
6     std::string line;
7     while (std::getline(in, line)) {
8         std::stringstream ss(line);
9         std::string number, brand, owner;
10        std::getline(ss, number, ';');
11        std::getline(ss, brand, ';');
12        std::getline(ss, owner);
13        Car car{};
14        strncpy(car.number, number.c_str(), sizeof car.number - 1);
15        strncpy(car.brand, brand.c_str(), sizeof car.brand - 1);
16        strncpy(car.owner, owner.c_str(), sizeof car.owner - 1);
17        out.write((const char*)&car, sizeof car);
18    }
19 }

```

```

1 // Предусловие: file открыт в двоичном режиме.
2 // Постусловие: возвращено число записей в файле.
3 long recordCount(std::ifstream& file) {
4     file.clear();
5     file.seekg(0, std::ios::end);
6     return (long)file.tellg() / (long)sizeof(Car);
7 }

```

```

1 // Предусловие: file открыт в двоичном режиме, 0 ≤ index <
recordCount(file).
2 // Постусловие: возвращена запись с порядковым номером index (прямой
доступ).
3 Car readRecord(std::ifstream& file, long index) {

```

```

4     Car car{};
5     file.clear();
6     file.seekg(index * (long)sizeof(Car));
7     file.read((char*)&car, sizeof car);
8     return car;
9 }

```

```

1 // Предусловие: car - заполненная запись.
2 // Постусловие: поля записи выведены в одну строку в std::cout.
3 void printCar(const Car& car) {
4     std::cout << car.number << " | " << car.brand << " | " << car.owner
<< '\n';
5 }

```

Главная программа (main.cpp) выполняет все три задания:

```

1 #include <chrono>
2 #include <cstdio>
3 #include <iostream>
4
5 #include "car.h"
6
7 using Clock = std::chrono::steady_clock;
8
9 // Среднее время выполнения f за repeats запусков, мкс
10 template <class F>
11 double measure(F f, int repeats) {
12     auto start = Clock::now();
13     for (int i = 0; i < repeats; i++) f();
14     std::chrono::duration<double, std::micro> d = Clock::now() -
start;
15     return d.count() / repeats;
16 }
17
18 std::string makeFile(int n) {
19     std::string base = "data/cars_" + std::to_string(n);
20     generateTextFile(base + ".txt", n, n);
21     textToBinary(base + ".txt", base + ".bin");
22     return base + ".bin";
23 }
24
25 int main(int argc, char** argv) {
26     const char* missing = "X000X00";
27     std::string path = makeFile(100);
28     std::ifstream file(path, std::ios::binary);
29
30     std::cout << "= Задание 1 =\n";
31     std::cout << "Файл " << path << ": записей " << recordCount(file)

```

```

32         << ", размер записи " << sizeof(Car) << " байт\n";
33     for (long i : {0L, 1L, 2L, 3L, 4L, 99L}) {
34         std::cout << '[' << i << "] ";
35         printCar(readRecord(file, i));
36     }
37
38     // ключ задается аргументом, иначе берется ключ записи с номером
57
39     std::string key = argc > 1 ? argv[1] : readRecord(file,
57).number;
40
41     std::cout << "\n= Задание 2 =\n";
42     for (const char* k : {key.c_str(), missing}) {
43         Car car;
44         long pos = linearSearch(file, k, car);
45         std::cout << "Ключ " << k << ": ";
46         if (pos < 0) std::cout << "не найден\n";
47         else {
48             std::cout << "запись " << pos << " → ";
49             printCar(car);
50         }
51     }
52
53     std::cout << "\n= Задание 3 =\n";
54     auto table = buildTable(file);
55     std::cout << "Таблица (первые 3 из " << table.size() << "):\n";
56     for (int i = 0; i < 3; i++)
57         std::cout << " " << table[i].number << " → " <<
table[i].offset << '\n';
58     for (const char* k : {key.c_str(), missing}) {
59         long offset = fibonacciSearch(table, k);
60         std::cout << "Ключ " << k << ": ";
61         if (offset < 0) std::cout << "не найден\n";
62         else {
63             std::cout << "смещение " << offset << " → ";
64             printCar(readByOffset(file, offset));
65         }
66     }
67
68     std::cout << "\n= Замеры времени (ключ последней записи файла),
мкс =\n";
69     std::cout << "          N          линейный          Фибоначчи
Фибоначчи+чтение          таблица\n";
70     for (int n : {100, 1000, 10000}) {
71         std::ifstream f(makeFile(n), std::ios::binary);
72         std::string last = readRecord(f, n - 1).number;
73         Car car;
74         auto t = buildTable(f);

```

```

75         double tLinear = measure([&] { linearSearch(f, last.c_str(),
car); }, 200);
76         double tFib = measure([&] { fibonacciSearch(t,
last.c_str()); }, 100000);
77         double tFibRead = measure([&] { readByOffset(f,
fibonacciSearch(t, last.c_str())); }, 10000);
78         double tBuild = measure([&] { buildTable(f); }, 50);
79         std::printf("%8d %12.3f %14.3f %17.3f %11.3f\n", n, tLinear,
tFib, tFibRead, tBuild);
80     }
81 }
82

```

1.4. Тестирование для 100 записей

```

1  Файл data/cars_100.bin: записей 100, размер записи 73 байт
2  [0] P799XM53 | Toyota | Сидоров С.М.
3  [1] E453YC79 | Haval | Кузнецов С.М.
4  [2] M983AA33 | BMW | Кузнецов А.А.
5  [3] 0669TK29 | Lada | Козлов П.В.
6  [4] T317HX43 | Geely | Кузнецов П.В.
7  [99] K192KH55 | Haval | Сидоров Д.Н.

```

Размер файла `cars_100.bin` — $7300 \text{ байт} = 100 \cdot 73$.

2. Задание 2

2.1. Постановка задачи

Найти запись по ключу в двоичном файле линейным поиском, оценить время поиска на файлах из 100, 1000, 10 000 записей.

2.2. Алгоритм

```

1  ЛинейныйПоиск(файл, ключ):
2      i ← 0
3      пока из файла прочитана очередная запись r:
4          если r.номер = ключ:
5              вернуть (i, r)
6          i ← i + 1
7      вернуть -1

```

2.3. Код функции поиска

```

1  // Предусловие: file открыт в двоичном режиме, key - строка-ключ.
2  // Постусловие: если запись с ключом key есть, она помещена в result и

```

```

возвращен
3 // ее порядковый номер; иначе возвращено -1.
4 long linearSearch(std::ifstream& file, const char* key, Car& result) {
5     file.clear();
6     file.seekg(0);
7     Car car;
8     for (long i = 0; file.read((char*)&car, sizeof car); i++) {
9         if (strcmp(car.number, key) == 0) {
10             result = car;
11             return i;
12         }
13     }
14     return -1;
15 }

```

2.4. Код программы

Функция вызывается из `main.cpp` (блок «Задание 2», п. 1.3).

2.5. Тестирование для 100 записей

- 1 Ключ K453AX43: запись 57 → K453AX43 | Skoda | Сидоров А.А.
- 2 Ключ X000XX00: не найден

2.6. Замеры времени

Искался ключ последней записи файла (худший случай), время усреднено по 200 запускам.

Таблица 2.1 — Время линейного поиска в файле

Число записей	Время, мкс
100	8,24
1000	66,84
10 000	685,43

3. Задание 3

3.1. Постановка задачи

Ускорить поиск в файле с помощью таблицы в оперативной памяти, содержащей ключ и смещение записи в файле. Поиск в таблице — поиск Фибоначчи. Запись по найденному смещению считывается прямым доступом. Оценить время поиска на файлах из 100, 1000, 10 000 записей.

3.2. Доступ к записи посредством таблицы

Элемент таблицы:

```
1 struct IndexEntry {
2     char number[9];
3     long offset;
4 };
```

Ссылка в таблице — смещение (в байтах) начала записи от начала файла. Таблица строится за один проход по файлу и сортируется по ключу (поиск Фибоначчи требует упорядоченности). Для доступа к записи используются средства `std::ifstream`: `seekg(offset)` устанавливает позицию чтения, `read` считывает `sizeof(Car)` байт в структуру.

3.3. Алгоритм

Поиск Фибоначчи делит отрезок не пополам, а в пропорции соседних чисел Фибоначчи; индексы вычисляются только сложением и вычитанием.

```
1 ПоискФибоначчи(T[0..n-1], ключ):
2     F2 ← 0; F1 ← 1; F ← 1 // F(k-2), F(k-1), F(k)
3     пока F < n: F2 ← F1; F1 ← F; F ← F1 + F2
4     сдвиг ← -1
5     пока F > 1:
6         i ← min(сдвиг + F2, n - 1)
7         если T[i].ключ = ключ: вернуть T[i].смещение
8         если T[i].ключ < ключ: // правая часть, длина F(k-1)
9             F ← F1; F1 ← F2; F2 ← F - F1; сдвиг ← i
10        иначе: // левая часть, длина F(k-2)
11            F ← F2; F1 ← F1 - F2; F2 ← F - F1
12        если F1 = 1 и сдвиг + 1 < n и T[сдвиг + 1].ключ = ключ:
13            вернуть T[сдвиг + 1].смещение
14        вернуть -1
```

3.4. Код функций

```
1 // Предусловие: file открыт в двоичном режиме.
2 // Постусловие: возвращена таблица (ключ, смещение) всех записей файла,
3 // упорядоченная по возрастанию ключа.
4 std::vector<IndexEntry> buildTable(std::ifstream& file) {
5     std::vector<IndexEntry> table;
6     file.clear();
7     file.seekg(0);
8     Car car;
9     for (long offset = 0; file.read((char*)&car, sizeof car); offset +=
10         sizeof car) {
```

```

10     IndexEntry e;
11     memcpy(e.number, car.number, sizeof e.number);
12     e.offset = offset;
13     table.push_back(e);
14 }
15 std::sort(table.begin(), table.end(), [](const IndexEntry& a,
const IndexEntry& b) {
16     return strcmp(a.number, b.number) < 0;
17 });
18 return table;
19 }

```

```

1 // Предусловие: table упорядочена по возрастанию ключа.
2 // Постусловие: возвращено смещение записи с ключом key в файле или -1,
3 // если ключа в таблице нет.
4 long fibonacciSearch(const std::vector<IndexEntry>& table, const char*
key) {
5     long n = table.size();
6     long f2 = 0, f1 = 1, f = 1; // F(k-2), F(k-1), F(k)
7     while (f < n) {
8         f2 = f1;
9         f1 = f;
10        f = f1 + f2;
11    }
12    long shift = -1; // элементы с индексом ≤ shift меньше key
13    while (f > 1) {
14        long i = std::min(shift + f2, n - 1);
15        int cmp = strcmp(table[i].number, key);
16        if (cmp == 0) return table[i].offset;
17        if (cmp < 0) { // ключ правее: отрезок длины F(k-1)
18            f = f1;
19            f1 = f2;
20            f2 = f - f1;
21            shift = i;
22        } else { // ключ левее: отрезок длины F(k-2)
23            f = f2;
24            f1 = f1 - f2;
25            f2 = f - f1;
26        }
27    }
28    if (f1 == 1 && shift + 1 < n && strcmp(table[shift + 1].number,
key) == 0)
29        return table[shift + 1].offset;
30    return -1;
31 }

```

```

1 // Предусловие: file открыт в двоичном режиме, offset - смещение начала
  записи.
2 // Постусловие: возвращена запись, расположенная по смещению offset.
3 Car readByOffset(std::ifstream& file, long offset) {
4     Car car{};
5     file.clear();
6     file.seekg(offset);
7     file.read((char*)&car, sizeof car);
8     return car;
9 }

```

3.5. Код программы

Функции вызываются из `main.cpp` (блок «Задание 3», п. 1.3).

3.6. Тестирование для 100 записей

```

1 Таблица (первые 3 из 100):
2   A235NM76 → 5986
3   A382PC34 → 2993
4   A534Y062 → 1168
5 Ключ K453AX43: смещение 4161 → K453AX43 | Skoda | Сидоров А.А.
6 Ключ X000XX00: не найден

```

Смещение $4161 = 57 \cdot 73$ — та же запись, что найдена линейным поиском.

3.7. Замеры времени

Искался ключ последней записи файла. Время поиска в таблице усреднено по 100 000 запускам, поиска с чтением записи — по 10 000, построения таблицы — по 50.

Таблица 3.1 — Время поиска Фибоначчи с таблицей смещений

Число записей	Поиск в таблице, мкс	Поиск + чтение записи, мкс	Построение таб- лицы, мкс
100	0,048	2,55	15,86
1000	0,090	2,59	170,15
10 000	0,103	2,63	3696,18

4. Анализ эффективности

Линейный поиск в файле имеет сложность $O(n)$: каждая запись считывается с диска и сравнивается с ключом. Время растёт пропорционально объёму

файла ($\approx$ в 10 раз при росте n в 10 раз). Достоинство — не требует упорядоченности и дополнительной памяти.

Поиск Фибоначчи в таблице выполняет $O(\log n)$ сравнений: для 10 000 записей — не более ≈ 20 , поэтому время поиска в таблице почти не зависит от n . Общее время определяется одним обращением к файлу по смещению ($\approx 2,6$ мкс) и для 10 000 записей в ≈ 260 раз меньше, чем у линейного поиска.

Плата за это — построение таблицы ($O(n \log n)$ из-за сортировки) и память: `sizeof(IndexEntry) = 24` байта на запись (9 байт ключа, 7 байт выравнивания, 8 байт смещения). Построение таблицы для 10 000 записей занимает время порядка пяти линейных поисков, поэтому таблица выгодна при многократном поиске в одном файле.

5. Выводы

Реализованы создание двоичного файла записей с прямым доступом, линейный поиск в файле и поиск Фибоначчи по таблице смещений в оперативной памяти. Замеры подтвердили линейный рост времени линейного поиска и логарифмический — поиска Фибоначчи; при многократных обращениях к файлу поиск по упорядоченной таблице смещений существенно эффективнее.